

Normalizing Flows and Diffusion Models

Overview

- outline the basic principles of
 1. Generative Modeling
 2. Normalizing Flows
 3. Diffusion Models

Async

**Introduction to
Generative
Modeling**

- assume that we have samples from some distribution p_x defined on $\mathbb{R}_d$
- what we want:
 - design a model that can efficiently sample from p_x
 - optionally: evaluate the likelihood $p_x(x)$ of a sample x
- uses:
 - generate novel samples
 - anomaly detection
- common methods:
 - transformers
 - VAEs
 - GANs
 - normalizing flows
 - diffusion models

What are Normalizing Flows?

- normalizing flows (NFs)

$$z \sim p_z \rightarrow f_1 \rightarrow f_2 \dots f_n \rightarrow x \sim p_x$$

$$f = f_n \circ \dots \circ f_2 \circ f_1$$

- for each layer f_i
 - efficiently compute its inverse
 - efficiently compute its Jacobian determinant

- example:

- invertible element-wise operations

$$\begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} \rightarrow \begin{bmatrix} \sigma(x_1) \\ \sigma(x_2) \\ \sigma(x_3) \\ \sigma(x_n) \end{bmatrix}$$

- the layer is invertible since the element-wise operation is invertible
- the layer's Jacobian is a diagonal matrix, so the determinant is the product of the diagonal

- example:

- shuffling layers

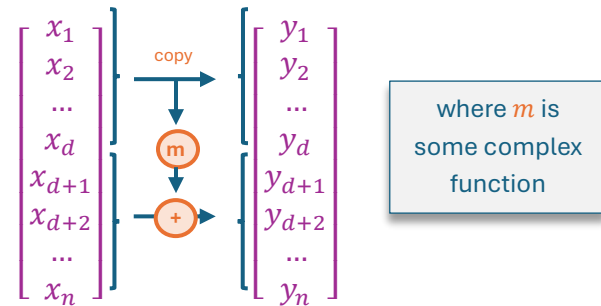
$$\begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_d \\ x_{d+1} \\ x_{d+2} \\ \dots \\ x_n \end{bmatrix} \rightarrow \begin{bmatrix} x_{d+1} \\ x_{d+2} \\ \dots \\ x_n \\ x_1 \\ x_2 \\ \dots \\ x_{n-1} \end{bmatrix}$$

- the layer is invertible since we can shuffle the elements

- the layer's Jacobian is a permutation matrix that has a determinant of 1

- example:

- coupling layers



- the layer is invertible since

$$x_{1:d} = y_{1:d} \text{ and } x_{d+1:n} = y_{d+1:n} - m(x_{1:d})$$

- the Jacobian is lower triangular, so the determinant is the product of the diagonal

- basic NF architectures interchange elementwise, shuffling, and coupling layers

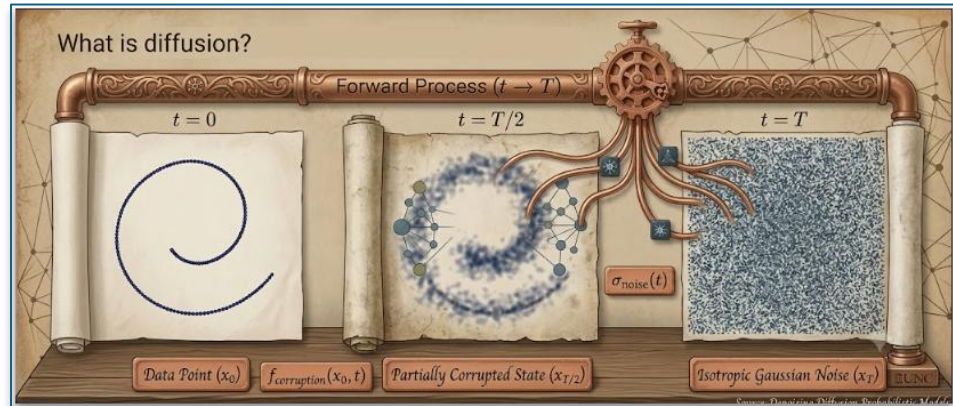
- NF pros:

- trained with maximum likelihood
- can generate new samples and evaluate likelihood of samples

- NF cons:

- NF architectures are restricted to using layers designed to be invertible and have traceable Jacobian determinant

Diffusion Processes



Gemini 3.6

- the diffusion process consists of several steps where we increasingly add small amounts of noise
- an ML model can then look at each step in this process and reverse the diffusion process to create an image from the noise
- diffusion can be defined as:

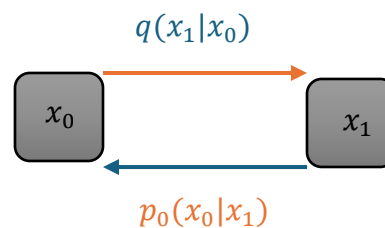
$$q(x_{t+1}|x_t) = N(\sqrt{1 - \beta}x_t, \beta I)$$

where, β is a small amount of Gaussian noise

- generating data using the diffusion process involves just reversing the process using learned parameters from a NN

Diffusion Models

- training diffusion models



- the process involves creating a NN that would maximize the probability of the observed data, essentially what all generative models do (exactly the case with normalizing flows)

- as is true with a lot of generative models, with diffusion models, rather than maximize the likelihood of the data, you maximize some lower bound of this data likelihood, or ELBO – evidence lower bound
- so, we are not trying to maximize the likelihood of the data directly, we are attempting to maximize the lower bound, guaranteeing that the likelihood is, at the least, higher than the lower bound

$$\begin{aligned}\log p_{\theta}(x) &= \log \int p_{\theta}(x_0 | x_1) p(x_1) dx_1 \\ &\geq E \left(\log p(x_1) + \log \frac{p_{\theta}(x_0 | x_1)}{q(x_1 | x_0)} \right)\end{aligned}$$

- this is then generalized to a multi-step process

$$\begin{aligned}\log p_{\theta}(x) &\leq E \left[\log p(x_T) + \log \frac{p_{\theta}(x_0 | x_1)}{q(x_1 | x_0)} \right. \\ &\quad \left. + \log \frac{p_{\theta}(x_1 | x_2)}{q(x_2 | x_1)} + \dots + \log \frac{p_{\theta}(x_{T-1} | x_T)}{q(x_T | x_{T-1})} \right]\end{aligned}$$

- after some manipulation, optimizing ELBO is equivalent to the following training
 - sample Gaussian noise ϵ , and use it to diffuse x_0 to x_t
 - use a NN to map (x_t, t) to a tensor with the same shape as x_t
 - compute loss as the MSE between the output and ϵ
- diffusion model architectures

```
# Pseudocode for a Single
Training Step

x = sample_from_dataset()
t = torch.randint()
noise =
    torch.randn_like(x)
```

```
x_t =
    noise_scheduler.add_noise(
        x, t, noise)
noise_pred =
    model(x_t, t)
loss = torch.mse(
    noise_pred, noise)
loss.backward()
```

```
# Pseudocode for Generating Samples
```

```
x_t = torch.randn(shape)
for t in range(T, 0):
    noise_pred = model(x_t, t)
    x_t = noise_scheduler.step(
        x_t, noise_pred)
```

- diffusion models are state-of-the-art for image generation
- can be generalized to class-conditional or text-conditioned image generation
- trained via ELBO instead of maximum likelihood
- no architectural constraints on NN shape or structure

Additional Readings

- [What Are Diffusion Models? \(Lillian Weng's Blog\)](#)
- [Diffusion Models as a Kind of VAE \(Angus Turner's Blog\)](#)
- [Denoising Diffusion Probabilistic Models \(Ho et. al. 2020\)](#)